

ADL Lite: An Event-First Operational Ontology for Multi-Agent Concept Consensus and Knowledge Graph Authoring

Allen Zhou^{1,*}

¹CEIEC AI Infrastructure

*Corresponding author

Abstract

The construction of knowledge graphs (KGs) increasingly relies on large language model (LLM) agents for discovery and authoring. However, existing ontology frameworks assume object-first design—concepts are mutable entities with stored properties—which creates fundamental tension with agentic workflows: multiple agents produce competing interpretations, status and confidence become stale fields, and centralized trust is required for validation.

We present ADL Lite, an event-first operational ontology that redefines concepts as append-only chains of typed events. Drawing on Wittgenstein’s *Tractatus* §1.1—“the world is the totality of facts, not of things”—ADL Lite treats every knowledge update as a new event rather than a mutation of existing state. The system provides a four-layer, Markdown-native syntax (L1: YAML, L2: Prose, L3: Structured Semantic Anchoring blocks, L4: Typed Action blocks). We introduce three core mechanisms: (1) Structured Semantic Anchoring (SSA), (2) a cryptographic EventChain with chained SHA-256 hashing for tamper evidence, and (3) a multi-agent consensus protocol with fork management and CRDT-inspired offline-first synchronization.

We evaluate ADL Lite through six controlled experiments plus a 10–100 agent distributed deployment simulation, validated by **350 automated tests** at 81% code coverage. The framework is implemented in Python 3.10+ with Pydantic v2, SQLite-backed hybrid memory, and a Feishu/Lark bridge. We provide a formal CRDT lattice proof for derived states, an RDF/OWL/PROV-O mapping with 100% information preservation, and comprehensive comparisons with Nanopublications, RDF-star, and Wikidata’s history model.

Keywords: operational ontology, multi-agent consensus, knowledge graph authoring, event-first design, structured semantic anchoring, cryptographic chain integrity, CRDT

1 Introduction

Knowledge graphs are increasingly authored not by individual experts but by fleets of large language model (LLM) agents operating in parallel [1, 2]. This shift raises a fundamental architectural question: *what is the right primitive for representing knowledge when the authors are autonomous agents?*

Existing ontology frameworks—RDF/OWL [3], property graphs [4], Pydantic schemas [5]—use object-first design. Concepts are mutable objects. This breaks down in multi-agent settings:

- (1) **State conflict:** Simultaneous writes produce race conditions.
- (2) **Auditability loss:** Reasons for state changes are lost.
- (3) **Trust centralization:** A coordinator must arbitrate writes.
- (4) **Training burden:** Agents need formal ontology languages.

We argue for **event-first** design, grounded in Wittgenstein [6]: “The world is the totality of facts, not of things.” A concept *is* its event chain. Status, confidence, validators are *computed*, not stored. This eliminates write-time race conditions; it reduces merge-time non-determinism to consistent ordering within timestamp buckets (§7.3).

ADL Lite provides: (1) a four-layer Markdown-native syntax, (2) Structured Semantic Anchoring (SSA), (3) a cryptographic EventChain with chained hash linking, (4) a multi-agent consensus protocol with fork management, and (5) a reference implementation with 350 automated tests.

Research questions:

- **RQ1 (Concurrency):** Can event-first design eliminate race conditions?
- **RQ2 (Ambiguity):** Does SSA reduce referential ambiguity?
- **RQ3 (Consensus):** Is CRDT-inspired merge sufficient for offline-first consensus?

2 Related Work

2.1 Operational Ontology and Event-First Design

Operational ontology has roots in process philosophy [7] and event-driven architectures [8]. PROV-O [9] models provenance as metadata *about* concepts. The Event Ontology [10] and SEM [11] model events as first-class but do not replace the concept-as-object paradigm.

2.2 KG Authoring by LLM Agents

Pan et al. [1], Trajanoska et al. [12], and Chen et al. [13] used LLMs for KG construction but relied on centralized orchestrators. ADL Lite targets the *authoring* phase—discovery and consensus—not completion or QA (FB15k, WN18RR, OAEI).

2.3 Consensus in Multi-Agent Systems

Consensus protocols for distributed systems [14, 15] and multi-agent systems [16–18] assume agents vote on propositions. ADL Lite’s model is inspired by CRDT principles [19]—append-only events are trivially mergeable—but derived states use last-writer-wins, not full lattice joins (§7.5).

2.4 Cryptographic Integrity in Knowledge Systems

Blockchain [20] and content-addressable storage [21] provide integrity but with overhead. ADL Lite uses lightweight SHA-256 hash chaining per concept.

2.5 Event-Centric RDF and Provenance Models

Nanopublications [22] model assertions as immutable, signed RDF graphs with provenance. ADL Lite differs by linking events into causal chains with predecessor hash binding, and by using Markdown-native authoring for LLM accessibility.

RDF-star [23] enables provenance annotations within RDF but does not prescribe lifecycle state machines or fork management.

Wikidata [24] records every edit as a revision—event-first thinking at scale. ADL Lite adds cryptographic chain integrity, structured fork resolution, and a portable authoring format.

RDF change sets [25] represent graph deltas. ADL Lite events carry semantic types (REGISTER, VALIDATE, FORK) beyond add/delete, embedding intent and reasoning.

3 The ADL Lite Framework

3.1 Design Philosophy

P1: Event-first. A concept *is* its EventChain. Properties are computed at query time.

P2: Agent-accessible syntax. Markdown is LLM-trained, human-readable, and supports layered structure.

P3: Concurrency-safe. Append-only + thread-safe locking + CRDT-inspired merge.

3.2 Four-Layer Syntax

L1: YAML Front Matter. Identity, type, scope. The L1 is a *snapshot* derived from the EventChain.

Listing 1: L1 YAML example

```

1 adl_type: discovery
2 adl_id: disc-capital-trap
3 status: provisional
4 confidence: 0.84
5 domain: financial_aml
6 scope: public

```

L2: Markdown Body. Free-form prose with [[Wiki Links]].

L3: Structured Semantic Anchoring. Typed blocks (adl:relation, adl:evidence, adl:seal) with machine-parseable YAML.

L4: Action Blocks. Typed actions with precondition-enforced side effects, validated against the YAML ontology registry.

3.3 EventChain: The Core Data Structure

Definition 3.1 (EventChain). An EventChain $C = (e_0, \dots, e_n)$ where $e_i = (\text{id}_i, \text{cid}, \tau_i, a_i, r_i, t_i, p_i, \text{prev}_i, h_i)$.

id_i is a full 128-bit UUID4 (32 hex characters, 2^{122} random bits). $h_i = \text{SHA-256}(\sigma(e_i) \parallel h_{i-1})$ for $i > 0$, $h_0 = \text{SHA-256}(\sigma(e_0))$, where σ serializes to canonical JSON sorted by key. The integrity invariant is $\forall i > 0 : \text{prev}_i = \text{id}_{i-1} \wedge h_i = \text{SHA-256}(\sigma(e_i) \parallel h_{i-1})$. Status derivation: $\text{status}(C) = f(\tau_k)$ where $k = \max\{i \mid \tau_i \in \mathcal{L}\}$ and $\mathcal{L}$ is the lifecycle event set.

3.4 Structured Semantic Anchoring (SSA)

SSA uses: (1) **Pronoun prohibition**—pattern-based detection, 29-sentence benchmark achieving **precision 0.812, recall 1.000, F1 0.897**; (2) **Typed slots** via Pydantic models; (3) **Ontology registry** with 10 predicates, 9 actions, 4 scope prefixes.

4 Multi-Agent Consensus Protocol

4.1 Lifecycle State Machine

provisional $\rightarrow$ validated/forked $\rightarrow$ deprecated $\rightarrow$ archived. Valid transitions enforced by the ontology registry.

4.2 Fork Management

Jaccard similarity on (relation, target) pairs: $\text{sim}(C_a, C_b) = |E_a \cap E_b| / |E_a \cup E_b|$. MERGE (≥ 0.90), PARALLEL (< 0.90), PRUNE (> 180 days idle). Threshold sensitivity: zero false positives across 0.70–0.95 sweep, F1=0.667 at default 0.90.

4.3 Offline-First Edge Synchronization

CRDT-inspired eventual consistency via `SyncManager.merge()`: collect, deduplicate by `event_id`, sort by (timestamp, hash), rebuild. `EdgeNode` buffers side effects when offline, drains on reconnect.

5 Implementation Architecture

The reference implementation (Python 3.10+, Pydantic v2, NetworkX) has 23 source modules across four layers: Parsing, Models, Semantics, Runtime.

Ontology Registry (`adl_core_ontology.yaml` v0.2): 5 classes, 10 predicates, 9 actions, 4 scope prefixes. Preconditions use Comparator enum (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS) with no `eval()`.

Hybrid Memory: Hot (in-memory, < 1ms), Warm (SQLite+NetworkX, 5–20ms), Cold (file-backed). Cascade filtering: status bitmap → type index → namespace trie → vector ANN → graph traversal.

Action Executor: 6-step pipeline (validate name, check params, evaluate preconditions, execute side effects, trigger transition, update log). Side effects via Protocol plugins.

Real-Time Watcher: Attaches to `append()`—laundering patterns, status transitions, frequency thresholds, amount alerts.

Feishu/Lark Bridge: Publish, announce, listen, sync-memory, dashboard—enterprise messaging as transport layer.

6 Experimental Evaluation

6.1 Experiment Design

Table 1: Experiments with research question mapping.

ID	Name	Validation Goal	RQ
E1	Chain integrity	Hash chain + tamper detection	RQ1
E2	Status derivation	Status from chain, not stored	RQ1
E3	Snapshot roundtrip	FrontMatter ↔ EventChain	RQ1
E4	Precondition enforce	Comparator pipeline integration	RQ2
E5	5-agent audit	Multi-agent validation + fork	RQ3
E6	IBM AML pipeline	Smart ontology discovery (P=1.000, R=0.500, F1=0.667)	RQ2

6.2 Results

All six experiments pass. **E1–E3**: hash chain correctly links events, tampering detected. **E4**: all 9 Comparator types validated. **E5**: 5 agents validate with distinct confidence scores. **E6**: IBM AML HI-Small_Trans (9,300 transactions), smart heuristic achieves precision 1.000, recall 0.500, F1 0.667.

6.3 Fork Threshold Sensitivity

Sweep 0.70–0.95 on 21 controlled concept pairs: **zero false positives across all thresholds**. Default 0.90 is conservative (F1=0.667, FPR=0.0). For higher recall, 0.75–0.80 achieves F1=0.857.

6.4 RDF/OWL/PROV-O Mapping

Turtle compiler: concepts $\rightarrow$ named graphs, relations $\rightarrow$ owl:ObjectProperty, evidence $\rightarrow$ prov:Entity. 100% information preservation on all 5 example documents (9/9 relations, 10/10 evidence).

6.5 Baseline Comparison

8-dimension comparison: ADL provides unique combination of Pydantic validation, SSA pronoun check, cryptographic audit trail (EventChain), ForkManager consensus, YAML registry, L2 prose, Markdown-native authoring—all in a single format.

6.6 Distributed Deployment (10–100 Agents)

Table 2: Distributed fault scenario results.

Scenario	Scale	Writes	Merged	Outcome
S1: Concurrent	10/50/100	30–300	100%	All events survive
S2: Partition	$3 \times (10/33/100)$	60–600	Deterministic	Order-independent
S3: Clock skew	10, ± 5 hr	20	Deterministic	Skew does not affect
S4: Adversarial	1 poison + 9 honest	68	Honest confidence 0.80	Survives attack

All chains pass `verify_integrity()`. Merge is deterministic regardless of entry order. Honest confidence survives adversarial poison events.

7 Discussion

7.1 Comparison with Object-First Ontologies

Table 3: Object-first vs. event-first.

Aspect	Object-First (typical)	Event-First (ADL Lite)
Concept repr.	Mutable object, stored properties	Append-only event chain
State derivation	Explicit writes to fields	Computed from chain
Audit trail	Requires explicit logging	Inherent (cryptographic)
Concurrency	Race conditions	Write-time: conflict-free
Trust model	Implicit in storage	Per-concept hash chain

7.2 Applicability to Semantic Web

ADL Lite bridges LLM authoring and RDF/OWL/PROV-O consumption: concepts $\rightarrow$ named graphs, events $\rightarrow$ annotations, relation blocks $\rightarrow$ OWL assertions.

7.3 Failure Modes and Adversarial Analysis

Timestamp collisions: SHA-256 secondary key for determinism; HLC/vector clocks for production. **Tampering:** chained hash provides evidence, not prevention; Ed25519 signing planned. **Chain poisoning:** watcher alerts signal but do not prevent; rate limiting future work. **SSA circumvention:** creative circumlocutions bypass pattern detection—confirmed experimentally. **Adversarial validation:** payload tampering detected, 100-event spam triggers alerts, 100-fork bombing handled correctly.

7.4 CRDT Formalization of Derived States

The derived state forms a join-semilattice $\mathcal{S} = (\text{Status} \times \mathbb{R}_{[0,1]} \times \mathbb{N} \times \mathbb{N}, \sqcup)$:

$$\text{merge}(s_1, s_2) = (\max(\text{status}_1, \text{status}_2), \max(\text{conf}_1, \text{conf}_2), \max(\text{val}_1, \text{val}_2), \max(\text{ev}_1, \text{ev}_2))$$

Status is lattice-ordered: PROVISIONAL $\sqsubset$ FORKED $\sqsubset$ VALIDATED $\sqsubset$ DEPRECATED $\sqsubset$ ARCHIVED.

Commutativity: $\text{merge}(A, B) = \text{merge}(B, A)$ — verified by enumeration. **Associativity:** $\text{merge}(\text{merge}(A, B), C) = \text{merge}(A, \text{merge}(B, C))$ — verified over 4^3 state triples. **Idempotence:** $\text{merge}(A, A) = A$. **Monotonicity:** $\text{apply}(s, e) \sqsupseteq s$. **Convergence:** under partition, $\text{merge}(s_A, s_B) \sqsupseteq s_A$ and $\sqsupseteq s_B$. Verified by 15 executable proofs (`tests/test_crdt_proofs.py`).

7.5 Limitations

- (1) **LWW semantics (addressed):** CRDT-correct alternative defined in §7.4.
- (2) **Single-concept granularity:** Cross-concept queries require ADLMemory layer.
- (3) **No formal semantics:** No description logic model theory.
- (4) **Scalability:** Hot layer $\approx 10\text{M}$ skeletons/node; $O(n)$ traversal for long chains.
- (5) **Lark coupling:** Platform-specific; SideEffect protocol enables alternatives.

7.6 Future Work

- (1) RDF/OWL compiler with named graph mapping.
- (2) Distributed EventChain (Kafka-backed).
- (3) LLM agent benchmark for ADL authoring.
- (4) Robust confidence aggregation (trust-weighted, per-agent credentials).
- (5) SHACL integration from ontology registry.
- (6) Cross-platform bridges (Slack, Teams, Discord).
- (7) Formal CRDT lattice proofs for full state.
- (8) Per-agent Ed25519 signatures + Merkle root anchoring.

8 Conclusion

We presented ADL Lite, an event-first operational ontology for multi-agent KG authoring. The key insight—concepts as append-only event chains—resolves fundamental tensions: race conditions become impossible, audit trails become inherent, trust becomes cryptographic. The four-layer Markdown-native syntax enables LLM agents; SSA constrains ambiguity; the consensus protocol with fork management and offline sync enables autonomous agent fleets. Validated by 6 experiments (6/6), distributed deployment at 10–100 agents, 350 tests at 81% coverage, CRDT lattice proofs, and a 100%-accurate RDF/OWL/PROV-O mapping.

Data Availability

Source code, ontology registry, experiments, and tests: <https://github.com/sunnyang1/adl-lite> (MIT License). Reproduce: `python -m experiments.runner all` and `pytest tests/ -v`.

Acknowledgments

[TO BE COMPLETED]

References

- [1] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [2] B. Chen, Z. Zhang, N. Beauchamp, J. Truong, Y. Wang, H. Yang, and L. Liu. Agentpoison: Red-teaming llm agents via memory poisoning. *arXiv preprint arXiv:2407.12784*, 2024.
- [3] Deborah L McGuinness and Frank Van Harmelen. Owl web ontology language overview. Recommendation, W3C, 2004.
- [4] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1):1–39, 2008.
- [5] Samuel Colvin et al. Pydantic: Data validation using python type hints. <https://github.com/pydantic/pydantic>, 2021.
- [6] Ludwig Wittgenstein. *Tractatus Logico-Philosophicus*. Routledge & Kegan Paul, 1922.
- [7] Alfred North Whitehead. *Process and Reality*. Free Press, 1929.
- [8] Martin Fowler. Event sourcing. <https://martinfowler.com/eaaDev/EventSourcing.html>, 2005.
- [9] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. Prov-o: The prov ontology. Recommendation, W3C, 2013.
- [10] Yves Raimond and Samer Abdallah. The event ontology. Technical report, Technical Report, 2007.
- [11] Willem Robert Van Hage, Véronique Malaisé, Roxane Segers, Laura Hollink, and Guus Schreiber. Design and use of the simple event model (sem). *Journal of Web Semantics*, 9(2):128–136, 2011.
- [12] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. Enhancing knowledge graph construction using large language models. *arXiv preprint arXiv:2305.04692*, 2023.
- [13] Z. Chen, H. Mao, H. Li, W. Jin, H. Wen, X. Wei, S. Wang, D. Yin, W. Fan, H. Liu, and J. Tang. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations*, 2024.
- [14] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.
- [15] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *USENIX Annual Technical Conference*, 2014.
- [16] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2nd edition, 2009.
- [17] Radu Jurca and Boi Faltings. An incentive compatible reputation mechanism. In *IEEE Conference on E-Commerce*, 2003.

- [18] Felix Brandt, Vincent Conitzer, Ulle Endriss, Jérôme Lang, and Ariel D Procaccia. *Handbook of Computational Social Choice*. Cambridge University Press, 2016.
- [19] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems (SSS)*, volume 6976 of *LNCS*, pages 386–400, 2011.
- [20] Allan Third and John Domingue. Linkchains: Exploring the space of decentralised trustworthy linked data. In *DeSemWeb@ISWC*, 2017.
- [21] Juan Benet. Ipfs - content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561*, 2014.
- [22] Paul Groth, Andrew Gibson, and Jan Velterop. The anatomy of a nanopublication. *Information Services & Use*, 30(1-2):51–56, 2010.
- [23] Olaf Hartig and Pierre-Antoine Champin. Rdf-star and sparql-star. Community group report, W3C, 2024.
- [24] Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.
- [25] Tim Berners-Lee and Dan Connolly. Delta: An ontology for the distribution of differences between rdf graphs. Note, W3C, 2004.